

Lab5 实验报告

一、思考题

1、设备操作与高速缓存

- 如果通过 kseg0 读写设备，那么对于设备的写入会缓存到 Cache 中。这是一种错误的行为，在实际编写代码的时候这么做会引发不可预知的问题。请思考：这么做这会引发什么问题？对于不同种类的设备（如我们提到的串口设备和IDE磁盘）的操作会有差异吗？可以从缓存的性质和缓存更新的策略来考虑。
 1. 设备直接修改内存（如 DMA 操作）时，Cache 中的数据没有被及时更新，如果我们通过 kseg0 来访问设备，那么就会导致缓存与内存内容不一致，CPU 读取到脏缓存数据而非设备更新的真实数据。
 2. 串口等 MMIO 设备寄存器读写敏感，顺序和时序必须严格保证，必须绕过缓存（通过 kseg1 或手动刷新缓存）；IDE 磁盘等块设备数据吞吐量大，通常通过 DMA 传输，寄存器操作较少，出错概率较小。

2、单个文件的最大体积、一个磁盘块最多存储的文件控制块及一个目录最多子文件

- 查找代码中的相关定义，试回答一个磁盘块中最多能存储多少个文件控制块？一个目录下最多能有多少个文件？我们的文件系统支持的单个文件最大为多大？

```
// user/include/fs.h
#define BLOCK_SIZE PAGE_SIZE
#define FILE2BLK (BLOCK_SIZE / sizeof(struct File))
#define NINDIRECT (BLOCK_SIZE / 4)
#define MAXFILESIZE (NINDIRECT * BLOCK_SIZE)
struct File {
    char f_name[MAXNAMELEN]; // filename
    uint32_t f_size; // file size in bytes
    uint32_t f_type; // file type
    uint32_t f_direct[NINDIRECT];
    uint32_t f_indirect;

    struct File *f_dir; // the pointer to the dir where this file is in, valid
    only in memory.
    char f_pad[FILE_STRUCT_SIZE - MAXNAMELEN - (3 + NINDIRECT) * 4 - sizeof(void
    *)];
} __attribute__((aligned(4), packed));
```

$FILE2BLK = BLOCK_SIZE / \text{sizeof}(\text{struct File}) = 4096 / 256 = 16$

$MAXFILESIZE = NINDIRECT * BLOCK_SIZE = 1024 * 4096 = 4MB$

$Max \text{ files per dir} = MAXFILESIZE / \text{sizeof}(\text{struct File}) = 16384$

一个磁盘块中最多能存储16个文件控制块；一个目录下最多能有16384个文件；我们的文件系统支持的单个文件最大为4MB。

3、磁盘最大容量

- 请思考，在满足磁盘块缓存的设计的前提下，我们实验使用的内核支持的最大磁盘大小是多少？

我们实验的 MOS 系统把整个磁盘的块和文件系统服务进程的内存空间进行了——映射。因此可用映射内存大小就是最大硬盘大小，所以最大磁盘大小为 1GB。

4、文件系统进程中宏定义理解

- 在本实验中，fs/serv.h、user/include/fs.h 等文件中出现了许多宏定义，试列举你认为较为重要的宏定义，同时进行解释，并描述其主要应用之处。

```
// fs/serv.h
#define PTE_DIRTY 0x0004 // 这个宏定义了一个标志位，用于页表项。当文件系统块缓存中的某个数据块被修改后，对应的页表项中会设置这个 DIRTY 位

#define SECT_SIZE 512 // 这个宏定义了磁盘扇区的大小为 512 字节。磁盘是存储数据的基本物理单元，读写操作通常以扇区为单位进行
#define SECT2BLK (BLOCK_SIZE / SECT_SIZE) // 这个宏定义了将磁盘扇区数转换为文件系统块所需的因子

#define DISKMAP 0x10000000 // 这个宏定义了一个虚拟内存地址，文件系统服务器会将从磁盘读取的块映射到这个地址开始的连续区域。每个磁盘块 n 在内存中会被映射到 DISKMAP + (n * BLOCK_SIZE) 这个地址

#define DISKMAX 0x40000000 // 这个宏定义了文件系统能够处理的最大磁盘容量，这里是 1GB (0x40000000 字节)
```

```
// user/include/fs.h
#define BLOCK_SIZE PAGE_SIZE // 这个宏定义了文件系统逻辑块的大小，并且强调它与系统的内存页大小相同。PAGE_SIZE 是操作系统内存管理的基本单元

#define MAXNAMELEN 128 // 这个宏定义了文件或目录名称的最大长度

#define MAXPATHLEN 1024 // 这个宏定义了文件或目录完整路径的最大长度

#define NDIRECT 10 // 这个宏定义了 struct File 结构中直接块指针的数量。每个直接指针可以直接指向文件的数据块

#define NINDIRECT (BLOCK_SIZE / 4) // 这个宏定义了一个间接块可以存储的块指针数量

#define MAXFILESIZE (NINDIRECT * BLOCK_SIZE) // 这个宏定义了通过一级间接寻址机制可以支持的最大文件大小

#define FILE_STRUCT_SIZE 256 // 这个宏定义了 struct File 结构的大小为 256 字节

#define FILE2BLK (BLOCK_SIZE / sizeof(struct File)) // 这个宏定义了一个文件系统块可以容纳多少个 File 结构体

// 文件类型
#define FTYPE_REG 0 // 普通文件
#define FTYPE_DIR 1 // 目录

#define FS_MAGIC 0x68286097 // 这个宏定义了一个魔数，用于标识文件系统的类型。当文件系统被挂载时，操作系统会检查磁盘上的这个魔数，以确认它是一个可以识别的文件系统
```

5、文件描述符与fork 函数

- 在Lab4“系统调用与fork”的实验中我们实现了极为重要的fork函数。那么fork前后的父子进程是否会共享文件描述符和定位指针呢？请在完成上述练习的基础上编写一个程序进行验证。

显然，fork() 之后，父子进程会共享文件描述符，并且这些文件描述符会指向相同的文件句柄。这意味着它们共享同一个文件偏移量（定位指针）。

```
// program
int r, fdnum, n;
char buf[200];
fdnum = open("/newmotd", O_RDWR);
if ((r = fork()) == 0) {
    n = read(fdnum, buf, 5);
    writef("[child] buffer is '%s'\n", buf);
} else {
    n = read(fdnum, buf, 5);
    writef("[father] buffer is '%s'\n", buf);
}
// output
[father] buffer is 'This '
[child] buffer is 'is a '
```

6、文件系统用户接口中的结构体

- 请解释 File, Fd, Filefd 结构体及其各个域的作用。比如各个结构体会在哪些过程中被使用，是否对应磁盘上的物理实体还是单纯的内存数据等。说明形式自定，要求简洁明了，可大致勾勒出文件系统数据结构与物理实体的对应关系与设计框架。

```
/*
file_open, file_create, file_get_block, file_set_size, file_close, file_remove
等文件系统操作都会使用或操作 File 结构体。
部分对应磁盘上的元数据（文件名、大小、类型、块指针），内存中的 f_dir 指针是纯粹的内存数据结构。
*/
struct File {
    char f_name[MAXNAMELEN]; // filename
    uint32_t f_size; // file size in bytes
    uint32_t f_type; // file type
    uint32_t f_direct[NDIRECT];
    uint32_t f_indirect;

    struct File *f_dir; // the pointer to the dir where this file is in, valid
    only in memory.
    char f_pad[FILE_STRUCT_SIZE - MAXNAMELEN - (3 + NDIRECT) * 4 - sizeof(void
    *)];
} __attribute__((aligned(4), packed));
/*
open() 系统调用成功时，内核会创建一个 Fd 结构体，将其关联到打开的 File 结构体和用户进程。
后续的 read(), write(), lseek(), close() 等系统调用会通过文件描述符找到对应的 Fd 结构
体，并进行相应的操作。
本身是纯粹的内存数据结构，它通过指向 File 结构体来间接关联磁盘上的文件数据。
*/
struct Fd {
    u_int fd_dev_id;
    u_int fd_offset;
    u_int fd_omode;
};
/*
```

```
用户态程序通过文件 I/O 库（如 C 标准库中的 FILE* 类型和相关函数）操作文件时，这些库可能会使用 Filefd 或类似的结构来管理底层的文件描述符。  
本身是用户态的内存数据结构，它通过包含文件描述符来间接关联内核中的 Fd 结构体和磁盘上的文件数据  
*/  
struct Filefd {  
    struct Fd f_fd;  
    u_int f_fileid;  
    struct File f_file;  
};
```

file_open, file_create, file_get_block, file_set_size, file_close, file_remove 等文件系统操作都会使用或操作 File 结构体。部分对应磁盘上的元数据（文件名、大小、类型、块指针），内存中的 f_dir 指针是纯粹的内存数据结构。

7、解释时序图，思考进程间通信

- 图 5.9 中有多种不同形式的箭头，请解释这些不同箭头的差别，并思考我们的操作系统是如何实现对应类型的进程间通信的。

同步消息，用黑三角箭头搭配黑实线表示；返回消息，用开三角箭头搭配黑色虚线表示。

通信类型	实现方式
函数调用	用户空间函数跳转（无需内核介入）
进程创建	系统调用（如 env_create()）
消息传递 IPC	ipc_send() / ipc_recv()，通常通过内核消息队列
页映射共享	虚拟内存管理中的页表共享
阻塞等待	通过调度器和等待队列

二、难点分析

感觉本单元的实验题难度都不是很大，根据提示照猫画虎基本都能写出来，难点主要是理解文件系统的结构和用户进程与文件系统服务进程之间的协作。

MOS 的文件系统是以根目录为根节点的树状数据结构。节点的数据结构由文件控制块 FCB（File 结构体）定义。一个 File 结构体的大小为固定的 256 字节（BY2FILE）。

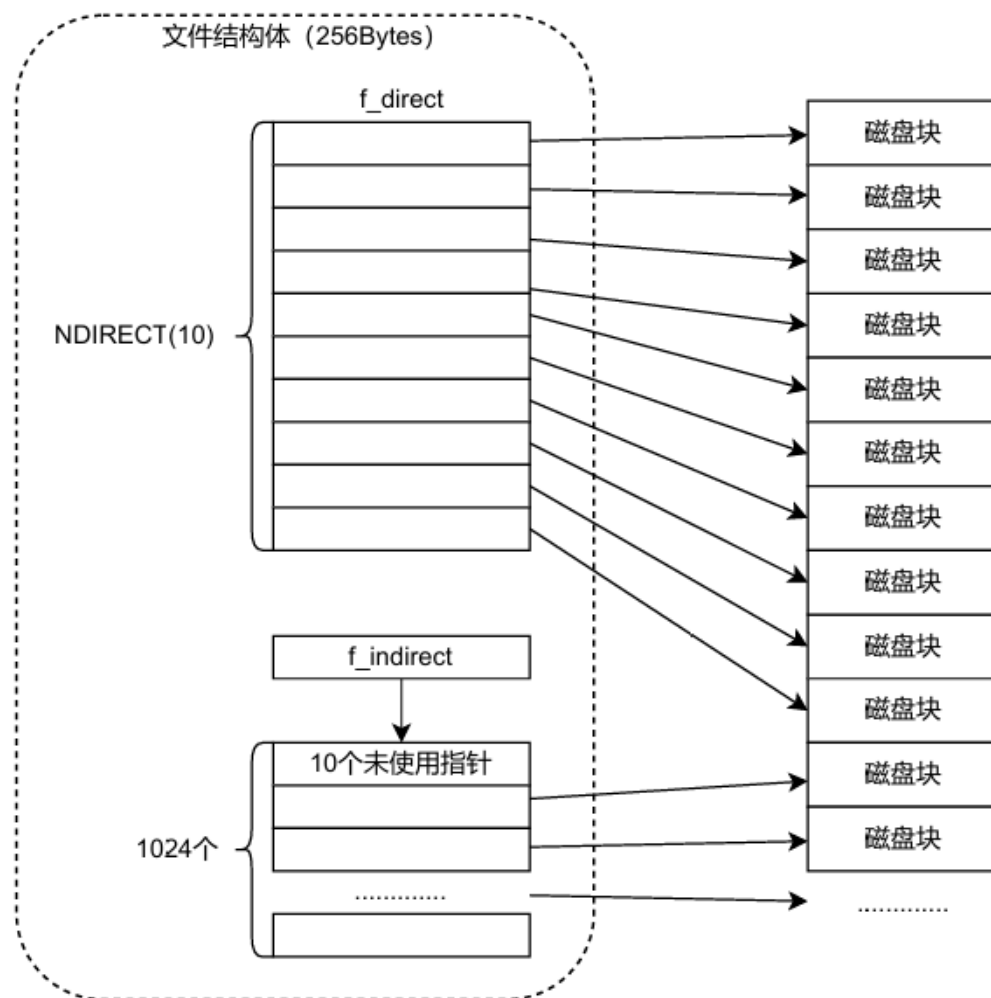
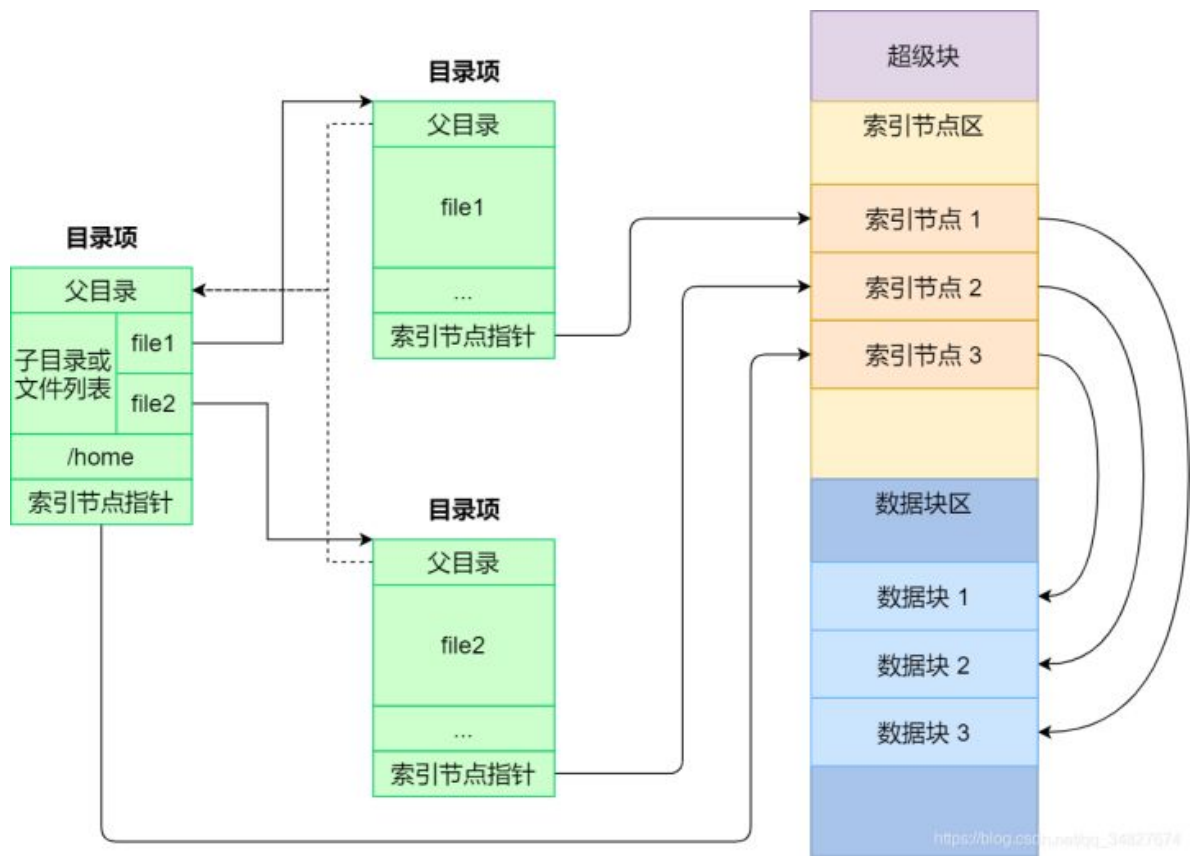
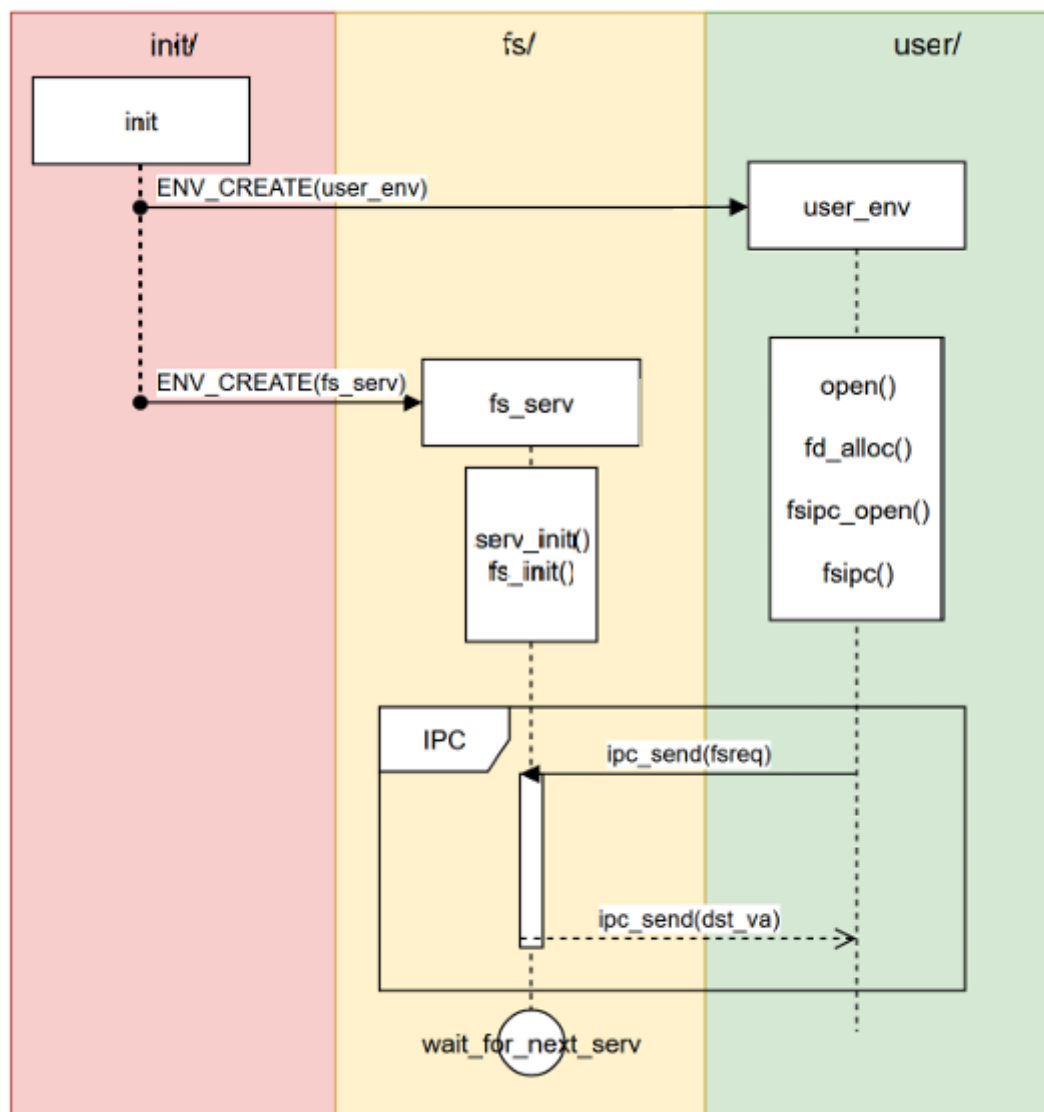


图 5.6: 文件控制块

MOS 操作系统的文件系统以独立服务进程（`fs_serv`）的形式运行，通过 IPC（`ipc_send/ipc_rcv`）接收并处理其他进程的文件操作请求。内核启动时创建文件系统服务进程（`fs_serv`），`fs_serv` 初始化通过 `serve_init` 准备文件打开记录表 `opentab` 并且掉个 `fs_init` 读取超级块获取磁盘信息，检查磁盘可读写性，并验证块位图。初始化完成后，`fs_serv` 进入 `serve` 循环，持续监听并处理其他进程的请求（如打开、读写文件等）。而用户进程只需通过 IPC 发送请求，由 `fs_serv` 完成实际文件操作。



三、实验体会

本次作业虽然实验难度不是很大，但是感觉自己的目光只局限在了局部的一个个小地方上，对整体的文件系统的操作还不是很理解，有种盲人摸象的感觉，很多实验代码也没有仔细去看。这次实验也是最后一次Lab了，回顾这几次实验的学习，虽然对操作系统有了一点理解，但是还是觉得自己的一些理解浮于表面。希望有时间可以从头在回顾一下这几次实验，构建一个更加宏观、具体的知识体系。

四、原创说明

- 部分内容参考[学长链接](#)
- 本文部分内容在LLM（大语言模型）的协助下完成